





# PADR

Malo Goossens

---

Dr. Laurent Philippe, M. Adrien Huygens

Master 1 en Architecture des Systèmes Informatiques
Année académique 2025-2026

# Table des matières


**1 Analyse du contexte** **5**

1.1 Introduction au protocole DNS . . . . . 5

1.2 Objectif du projet . . . . . 5

1.3 Périmètre et limites du système . . . . . 5

1.4 La contrainte de python . . . . . 5

1.5 Choix de la bibliothèque standard . . . . . 5

1.6 L'interopérabilité et le respect de la RFC 1035 . . . . . 6

**2 Analyse fonctionnelle** **7**

2.1 Vision globale du système . . . . . 7

2.2 User stories . . . . . 8

2.2.1 Épic 1 : L'Outil Client (CLI) . . . . . 9

2.2.2 Épic 2 : Le Serveur Récursif (Le Chef d'orchestre) . . . . . 10

2.2.3 Épic 3 : Le Serveur d'Autorité (La Base de données) . . . . . 11

2.2.4 Épic 4 : Tolérance aux Pannes & Réplication (Master / Slave) . . . . . 11

2.3 Épic 5 : Environnement, Qualité et Méthodologie (En tant que Dev) . . . . . 12

**3 Analyse technique** **13**

3.1 Encodage et Sérialisation des données . . . . . 13

3.1.1 La Forge des Messages . . . . . 13

3.1.2 Encodage et masques bit-à-bit . . . . . 15

3.1.3 Compression des noms et gestion de la section ADDITIONAL . . . . . 15

3.1.4 Encodage des enregistrements TXT . . . . . 15

3.2 Architecture réseau . . . . . 16

3.2.1 Stratégie Réseau (UDP vs TCP) . . . . . 16

3.2.2 Gestion du Fallback TCP . . . . . 16

3.2.3 Générisité de l'architecture . . . . . 17

3.3 Résolution DNS . . . . . 20

3.3.1 Parcours d'une requête récursive . . . . . 20
3.3.2 Gestion du Cache . . . . . 21
3.4 Chargement de zones . . . . . 22
3.4.1 Stockage JSON . . . . . 23
3.4.2 Rechargement à chaud et surveillance de fichier . . . . . 24
3.4.3 La Réplication Master / Slave et Rechargement . . . . . 24
3.5 Conception des classes . . . . . 26

**4 Implémentation** **26**
4.1 Arborescence du Projet . . . . . 26
4.2 Qualité et Tests (TDD) . . . . . 28
4.2.1 Tests unitaires . . . . . 28
4.2.2 Tests de bout en bout . . . . . 28
4.3 Bugs critiques identifiés et résolus . . . . . 29
4.4 Déploiement sous Docker . . . . . 30

**5 Analyse retrospective** **32**
5.1 Bilan des résultats . . . . . 32
5.2 Limites et perspectives d'améliorations . . . . . 32

## Bibliographie

[1] K. R. Steven, *TCP-IP Illustrated Volume 1*, vol. 1, ch. 11. Addison-Wesley, 2 ed., 2012.

# Introduction

# 1 Analyse du contexte

## 1.1 Introduction au protocole DNS

## 1.2 Objectif du projet

## 1.3 Périmètre et limites du système

Le périmètre de cette implémentation inclut la résolution itérative des enregistrements A (ipv4) et AAAA (ipv6), la gestion d'un cache local, le fallback vers TCP pour les réponses volumineuses, ainsi que la réplication des zones via AXFR et NOTIFY. Le système est conçu pour être interopérable avec les outils existants tels que dig et nslookup.

Le cahier des charges n'inclut donc pas la mise en œuvre de signatures DNSSEC, le chiffrement, le support des requêtes en IPv6 (même si les enregistrements AAAA sont supportés), ni l'extention EDNS0 permettant de négocier des tailles de trames supérieures à 512 octets en UDP.

L'architecture du système doit cependant faciliter l'ajout éventuel de ces fonctionnalités.

## 1.4 La contrainte de python

Le language de programmation Python est imposé par les consignes du projet pour la réalisation du programme. Python est un langage de haut niveau, dynamiquement typé, interprété et multiparadigmes, il est apprécié pour sa lisibilité et sa simplicité. Dans le cadre de ce projet, la présence des mécanismes orientés objet, de la gestion automatique de la mémoire et d'une bibliothèque standard riche (`struct`, `socket`, `threading`, `json`) sont d'une utilité considérable. Cependant, l'envoi de messages binaires et la gestion de sockets devant être implémentés manuellement, l'abstraction offerte par Python parfois devenir une gêne. Contrairement à des langages de plus bas niveau (comme le C++) qui permettent d'utiliser des structures avec des pointeurs pour lire les données binaires directement, Python demande un travail constant et explicite de sérialisation et désérialisation. Heureusement, le module `struct` permet de faire le pont entre les objets de haut niveau manipulés par le code et les trames binaires brutes exigées par le réseau.

## 1.5 Choix de la bibliothèque standard

Dans l'intérêt pédagogique du projet, il est interdit de déléguer l'implémentation de certaines parties du système à des bibliothèques tierces. Afin de ne pas risquer de manquer à cette contrainte, l'ensemble du projet a été développé en utilisant exclusivement la bibliothèque standard Python.

Hors du respect de la contrainte académique, ce choix présente de multiples bénéfices :

— **Portabilité et absence de dépendances :** Le projet peut être exécuté sur n'importe quelle

machine disposant d'une installation Python standard, sans nécessiter l'installation de modules supplémentaires.

— **Pérennité et stabilité à long terme :** Les bibliothèques tierces disponibles sur PyPi évoluent rapidement et peuvent déprécier certaines fonctions ou être complètement abandonnées ce qui crée une dette technique. À l'inverse, un projet basé sur la bibliothèque standard Python restera maintenable et fonctionnel sur le long terme.

— **Légèreté et optimisation :** Les bibliothèques externes introduisent des fonctions superflues et des dépendances inutiles. En se limitant à la bibliothèque standard, nous avons la garantie que le code n'inclut que le nécessaire.

— **Sécurité et maîtrise de la chaîne logistique :** Dans le contexte de la cybersécurité moderne, dépendre de multiples paquets externes expose le projet à des failles de type Supply Chain Attack. Restreindre le projet à la bibliothèque standard garantit que la surface d'attaque est limitée à Python lui-même.

## 1.6 L'interopérabilité et le respect de la RFC 1035

Afin de garantir l'interopérabilité avec les outils existants, le projet doit respecter le format de trame binaire défini par la RFC 1035. Le respect de cette norme permettra au système de communiquer correctement avec les serveurs DNS existants et d'assurer la compatibilité avec des outils tels que dig ou nslookup.

# 2 Analyse fonctionnelle

## 2.1 Vision globale du système

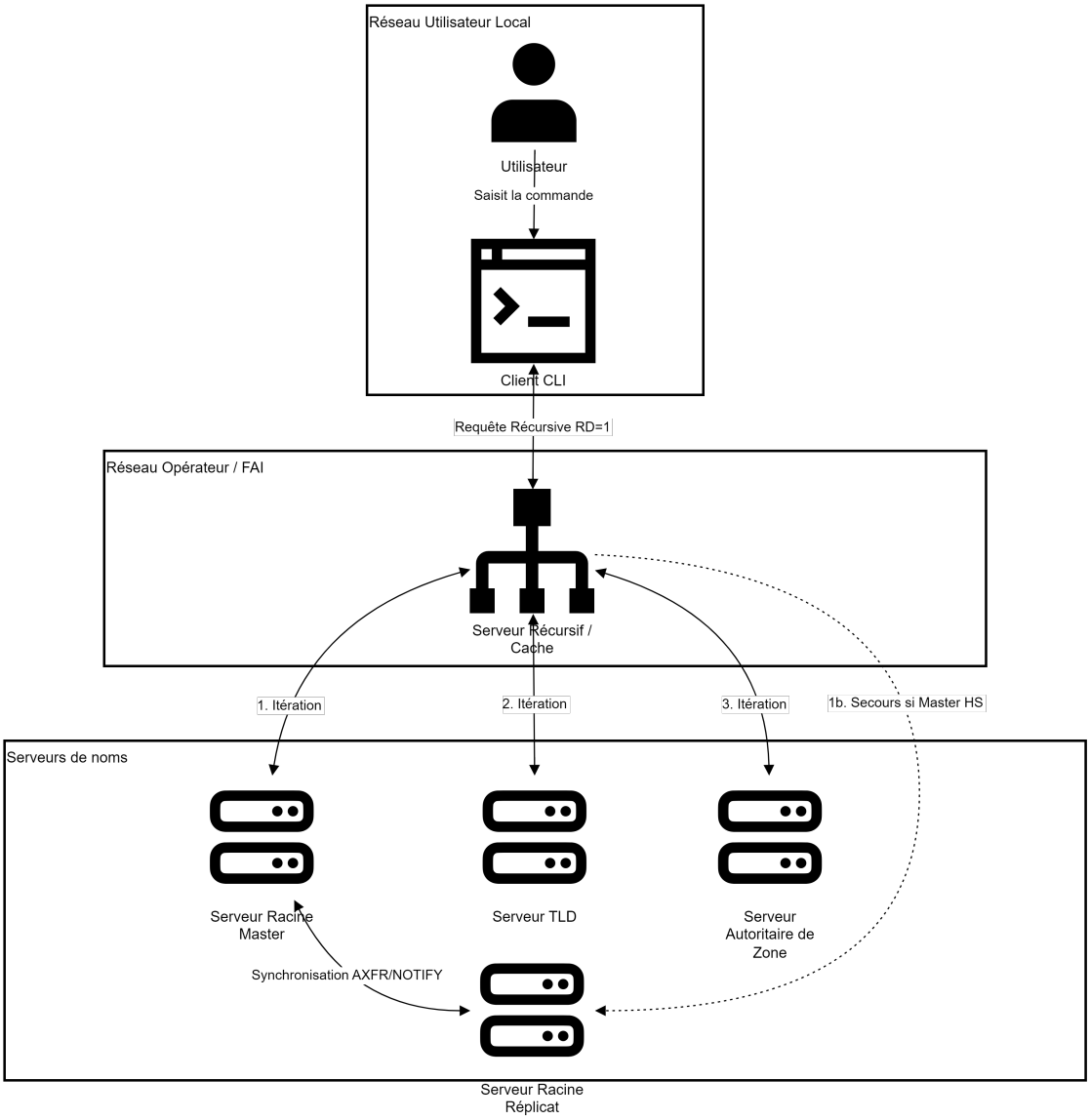



FIGURE 1 – Architecture globale des flux entre le Client, le Résolveur et la Hiérarchie DNS.

Le processus de résolution d'un nom de domaine s'articule autour des trois niveaux d'infrastructure présentés dans le schéma :

1. **Émission de la requête initiale (Réseau Utilisateur Local) :** L'utilisateur saisit une commande réseau (ex : `dig` ou le client Python). Le `Client CLI` génère une trame DNS en

activant le bit `RD=1` (*Recursion Desired*), signalant au résolveur qu'il souhaite obtenir une réponse finale complète sans effectuer lui-même les étapes itératives.

2. **Prise en charge par le résolveur (Réseau Opérateur / FAI) :** La requête parvient au `Serveur Récursif / Cache`. Ce dernier interroge en premier lieu son cache local :
   - **En cas de succès (*Cache Hit*) :** La réponse est directement retournée au client avec le bit `AA=0` et le TTL décrémenté.
   - **En cas d'échec (*Cache Miss*) :** Le résolveur initie la chaîne de résolution itérative vers la hiérarchie DNS.

3. **Cycle de résolution itérative (Serveurs de Noms) :**
   - **1. Première itération (Racine) :** Le résolveur interroge le `Serveur Racine Master` (ou bascule vers le serveur de secours répliqué si le master est hors service, selon la route *1b. Secours si Master HS*). Le serveur racine répond par une délégation vers les serveurs de noms du TLD concerné.
   - **2. Deuxième itération (TLD) :** Le résolveur contacte le `Serveur TLD` (ex : `.be`), qui lui retourne l'adresse IP et les *Glue Records* du serveur autoritaire responsable du domaine.
   - **3. Troisième itération (Autorité) :** Le résolveur interroge le `Serveur Autoritaire de Zone`. Ce dernier consulte sa zone locale et renvoie la réponse autoritaire finale (`AA=1`) contenant les enregistrements demandés (ex : enregistrement `A`).

4. **Mécanisme de réplication sous-jacent :** En arrière-plan, l'intégrité des données entre les serveurs primaires et secondaires est maintenue via le mécanisme de réplication Master/Slave basculant sur les protocoles `NOTIFY` (signalement de modification) et `AXFR` (transfert de zone complet).
5. **Finalisation et mise en cache :** Une fois la réponse enregistrée auprès du serveur d'autorité, le résolveur récursif met en cache le résultat pour la durée spécifiée par le TTL, puis transmet la trame de réponse finale au `Client CLI`.

## 2.2 User stories

L'aboutissement de l'analyse fonctionnelle est de dresser la liste des *User Stories* (US) et des *Épics* correspondants, qui décrivent les fonctionnalités attendues du système du point de vue de l'utilisateur final.

### 2.2.1 Épic 1 : L'Outil Client (CLI)

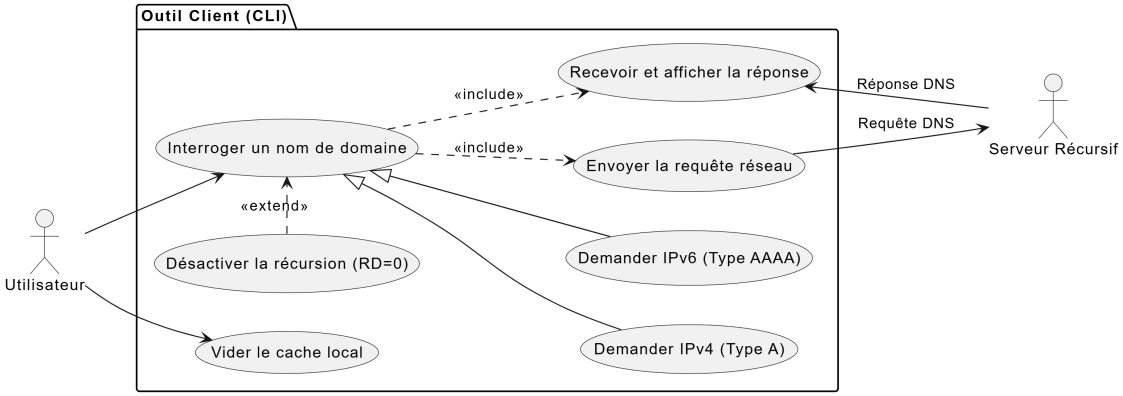



FIGURE 2 – Diagramme de cas d'utilisation du Client CLI.

— **US-1.1 :** En tant qu'utilisateur, je veux pouvoir tester toutes les fonctionnalités du système depuis un client CLI afin de pouvoir vérifier son bon fonctionnement.

— **US-1.2 :** En tant qu'utilisateur, je veux pouvoir résoudre un nom de domaine en adresse IPv4 (Type A) afin de pouvoir me connecter au serveur cible.

— **US-1.3 :** En tant qu'utilisateur, je veux pouvoir résoudre un nom de domaine en adresse IPv6 (Type AAAA) afin de pouvoir me connecter au serveur cible sur un réseau moderne.

— **US-1.4 :** En tant qu'utilisateur, je veux pouvoir envoyer un signal d'administration au serveur récursif afin de vider son cache à distance pour du diagnostic.

— **US-1.5 :** En tant qu'utilisateur, je veux pouvoir désactiver la récursion (‘RD=0‘) via une option du CLI afin d'interroger un serveur de manière ciblée pour du diagnostic.

— **US-1.6 :** En tant qu'utilisateur, je veux pouvoir utiliser la solution sur mon ordinateur personnel en me limitant à de la configuration réseau pour que mon ordinateur effectue les résolutions de façon transparente.

### 2.2.2 Épic 2 : Le Serveur Récursif (Le Chef d'orchestre)

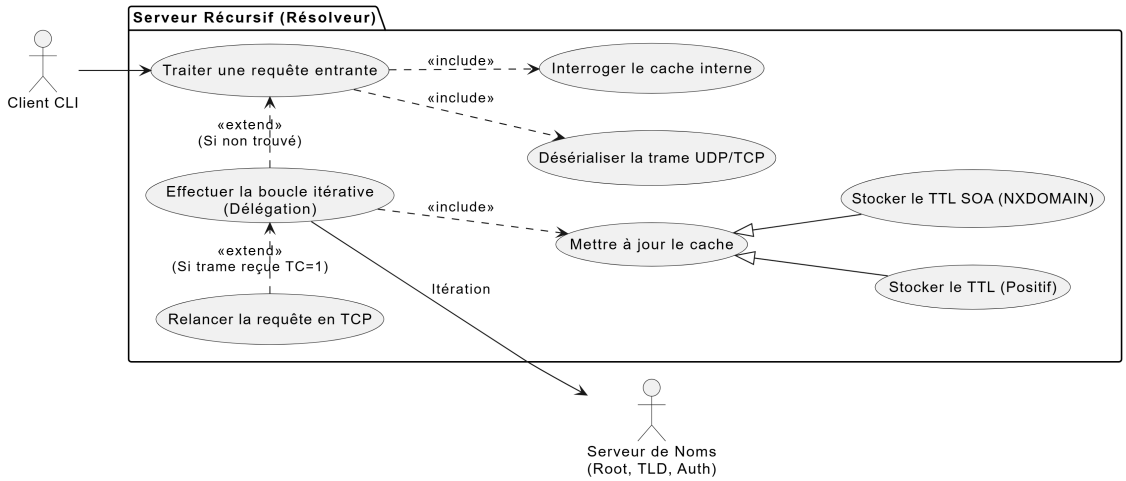



FIGURE 3 – Diagramme de cas d'utilisation du Résolveur Récursif.

— **US-2.1 :** En tant que résolveur, je veux écouter sur le port UDP 53 et désérialiser la trame binaire entrante afin d'extraire le nom de domaine et le type de requête demandés.

— **US-2.2 :** En tant que résolveur, je veux interroger mon cache interne avant d'émettre une requête sur le réseau afin d'accélérer drastiquement le temps de réponse.

— **US-2.3 :** En tant que résolveur, je veux implémenter la boucle de délégation (Root -> TLD -> Autorité) afin de trouver la réponse à la requête.

— **US-2.4 :** En tant que résolveur, je veux extraire le TTL des réponses positives (A/AAAA) et les stocker en mémoire afin de construire mon cache.

— **US-2.5 :** En tant que résolveur, je veux lire l'enregistrement SOA lors d'une erreur *NXDOMAIN* afin de mettre cette absence en cache (Negative Caching).

— **US-2.6 :** En tant que résolveur, si une réponse UDP contient le flag 'TC=1' (Tronqué), je veux relancer automatiquement ma requête via TCP afin d'obtenir la donnée complète.

— **US-2.7 :** En tant que résolveur, je veux intercepter les messages d'administration du client afin de vider instantanément ma structure de données de cache en RAM.

### 2.2.3 Épic 3 : Le Serveur d'Autorité (La Base de données)

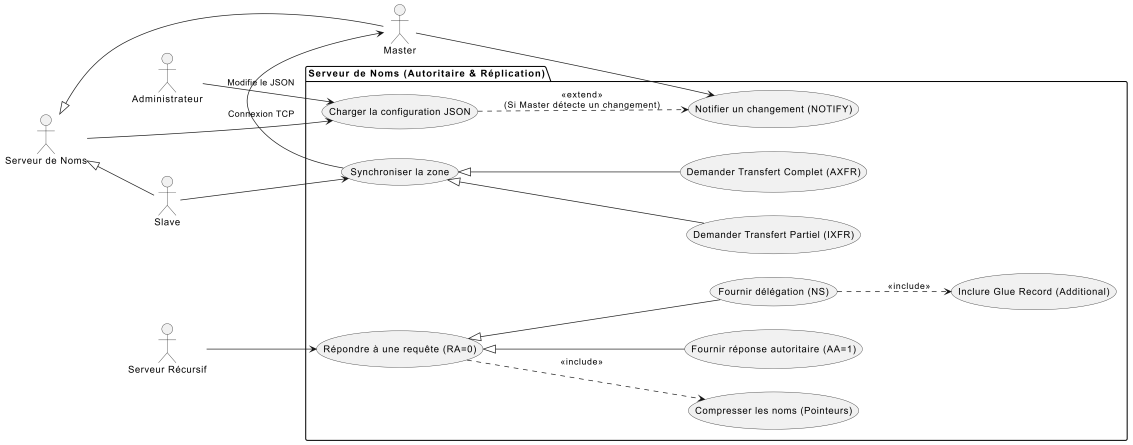



FIGURE 4 – Diagramme de cas d'utilisation du Serveur de Nom.

— **US-3.1 :** En tant que serveur de nom, je veux charger ma base de données (Zone) depuis un fichier JSON au démarrage afin de connaître les enregistrements sous ma responsabilité.

— **US-3.2 :** En tant que serveur de nom, si la requête correspond à une IP que je connais, je veux renvoyer une réponse autoritaire (‘AA=1‘) contenant la ou les IP (RRSets) dans la section *Answer*.

— **US-3.3 :** En tant que serveur de nom, si je ne gère qu'une portion du domaine (ex : Root ou TLD), je veux renvoyer une délégation (enregistrement NS) dans la section *Authority* afin que le serveur récursif puisse continuer sa recherche.

— **US-3.4 :** En tant que serveur de nom, lors d'une délégation, je veux inclure l'adresse IP du serveur délégué dans la section *Additional* (Glue Record) afin de faire gagner du temps au résolveur.

— **US-3.5 :** En tant que serveur de nom, je veux implémenter l'algorithme de compression (pointeur ‘11xxxxxx‘) lors de la création de ma réponse binaire afin de ne pas saturer la limite de 512 octets de l'UDP.

— **US-3.6 :** En tant qu'administrateur, je veux pouvoir déclencher le rechargement du fichier JSON "à chaud" (sans redémarrer le processus du serveur) afin d'ajouter ou modifier des enregistrements sans provoquer d'interruption de service.

### 2.2.4 Épic 4 : Tolérance aux Pannes & Réplication (Master / Slave)

— **US-4.1 :** En tant que serveur Racine Réplica (Slave), je veux me connecter en TCP au serveur Racine Master à mon démarrage et demander un transfert de zone complet (AXFR) afin d'obtenir la configuration initiale.

— **US-4.2 :** En tant que serveur Racine Master, je veux accepter les requêtes de type AXFR en

TCP et renvoyer l'intégralité de mon fichier de zone afin d'assurer la redondance requise.

— **US-4.3 :** En tant que serveur Racine Master, lorsque mon fichier JSON est modifié, je veux envoyer un message UDP (OpCode 4, NOTIFY) à mon Slave afin de le prévenir instantanément d'un changement.

— **US-4.4 :** En tant que serveur Racine Réplica (Slave), à la réception d'un NOTIFY ou après l'expiration de mon timer de Refresh, je veux comparer mon numéro de série (SOA) avec celui du Master afin de savoir si je dois me mettre à jour.

— **US-4.5 :** En tant que serveur Racine Réplica, si le numéro de série du Master est plus grand, je veux effectuer une requête de transfert incrémental (IXFR) en TCP pour télécharger uniquement les nouveaux enregistrements.

## 2.3 Épic 5 : Environnement, Qualité et Méthodologie (En tant que Dev)

— **US-5.1 :** En tant que développeur, je veux qu'un pipeline d'Intégration Continue exécute automatiquement mes tests unitaires à chaque push afin de valider la non-régression de mon code.

# 3 Analyse technique

## 3.1 Encodage et Sérialisation des données

### 3.1.1 La Forge des Messages

Le cœur du protocole DNS est son format de message permettant aux clients, résolveurs récursifs et serveurs de noms de communiquer. Il est donc essentiel de simplifier au maximum la lecture, la création et la (dé)sérialisation de ces messages qui seront utiliser par chaques composants du système.

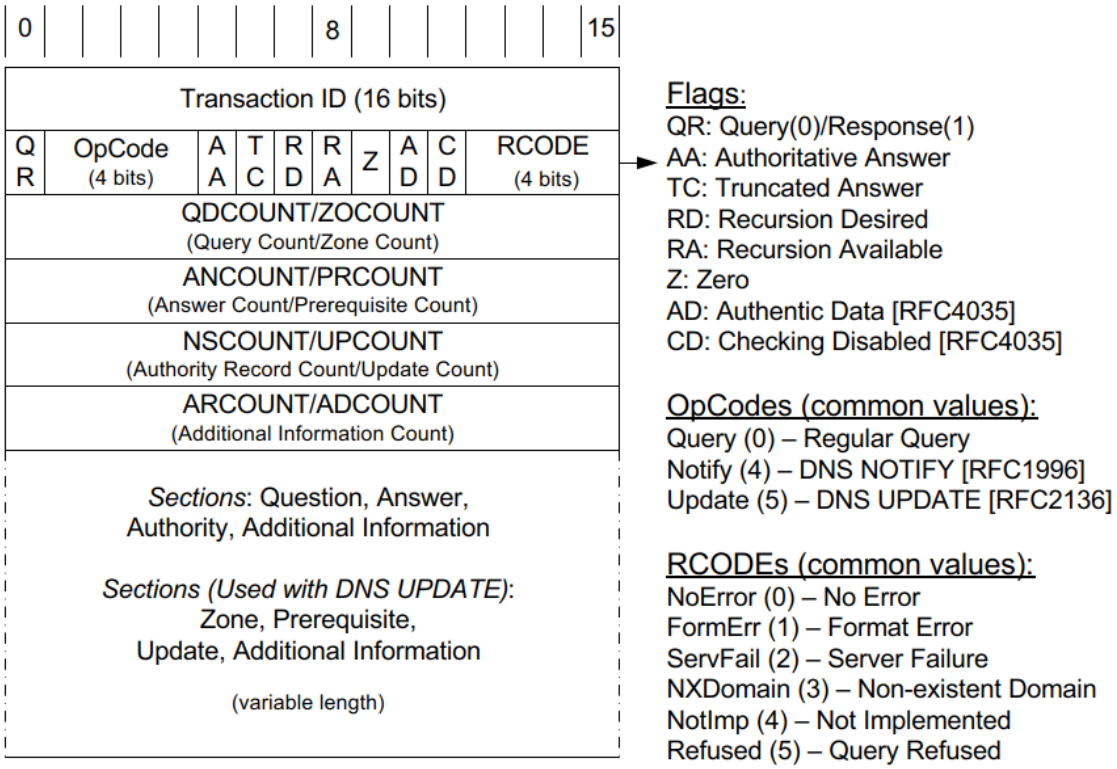



FIGURE 5 – Format des messages DNS [1].

Comme illustré, le contenu d'un message DNS peut être séparé en sept sections. Premièrement, un identifiant de transaction (encodé sur 16 bits) définit par le client et recopié par le serveur permet d'associer une requête à sa réponse. La section suivant est un tampon de 16 bits constituants des drapeaux (flags) qui permettent de donner des informations sur la nature et l'état de la requête. Ensuite, quatre nombres entiers (16 bits) donne le nombre d'entrées dans les sections suivantes, permettant ainsi de connaitre leur taille, ce qui est nécessaire pour les lire. Finalement, la section query (question) Les sections `question`, `answer`, `authority` et `additionnal` information, constituent le contenu du message. Elles permettent respectivement de connaitre le type de données attendu, la réponse du serveur, les serveurs détenant la réponse et des informations

complémentaires.

Les questions ont un format qui leur est propre, il est composé du nom de domaine concerné et du type de question qui est posé. Les autres champs utilisent tous un format appelé `Resource Record` pour représenter les données transmises. Ils sont composés d'un nom de domaine, de leur type (A, AAAA, NS, CNAME, TXT), de leur classe (toujours IN), du temps qu'ils doivent être conservés en cache (ttl) et de la donnée qu'ils contiennent au format binaire. Les noms de domaines sont compressés par un algorithme qui évite d'encoder plusieurs fois les mêmes labels dans le message.

Ainsi, quatre classes python seront utilisées pour rendre la manipulation des messages transparente. La classe `DNSMessage` contiendra les sept section présentés précédemment ainsi que les méthodes `pack` et `unpack` pour assurer la (dé)sérialisation. Les classes `DNSQuestion` et `ResourceRequest` seront utilisées par `DNSMessage` pour représenter son contenu. Le champ donné de `ResourceRecord` dépendant de son type, il est stocké au format binaire et est rendu accessible par une propriété qui le décode, à la demande, de la manière adéquate. Enfin, la classe `DomainName` centralise la compression.

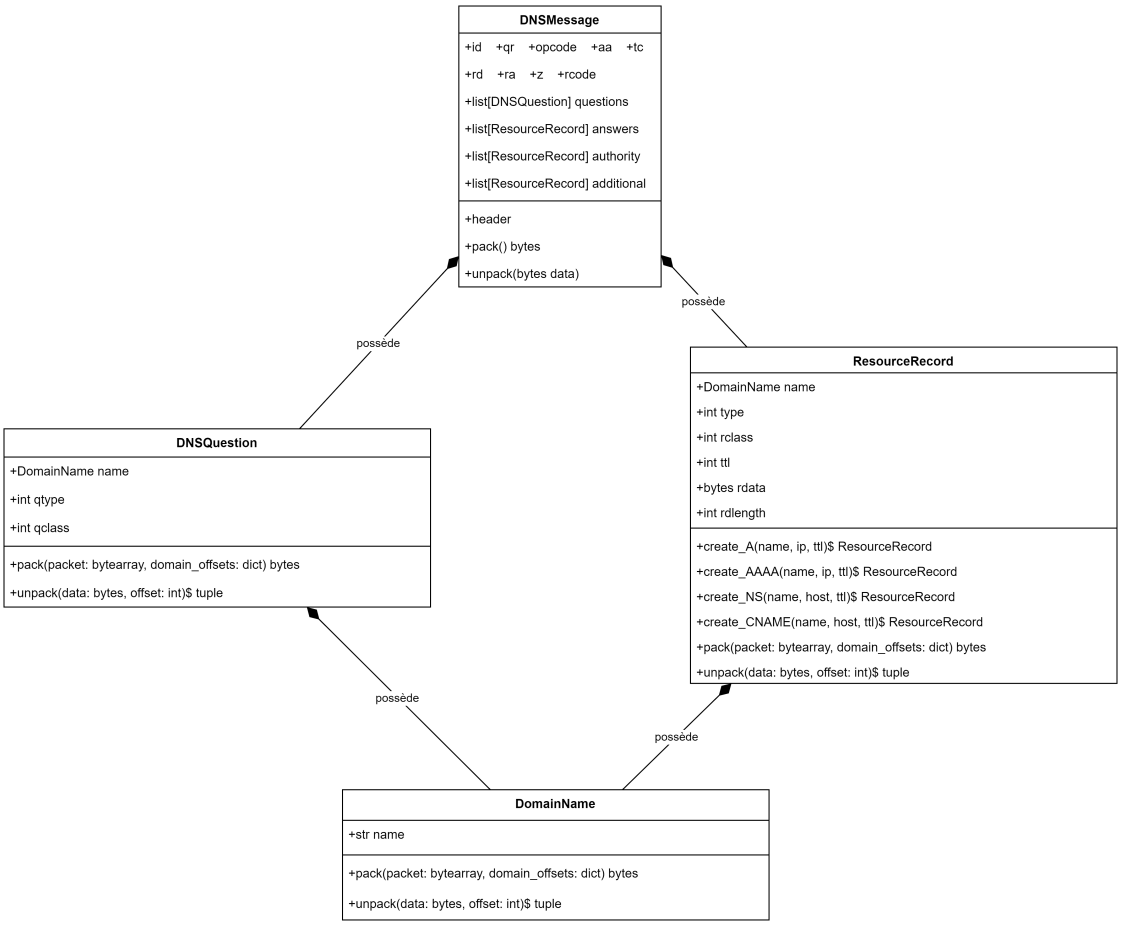



FIGURE 6 – Diagramme de classe de la Forge à messages.

### 3.1.2 Encodage et masques bit-à-bit

Envoyer un message sur le réseau demande un contrôle de la couche binaire. Python étant un langage de haut niveau, il est nécessaire de passer par le module `Struct` pour atteindre cette granularité

`Struct` permet de convertir des types de données Python en flux d'octets et inversement. Pour ce faire, il est nécessaire de définir un format de conversion. Dans le cas de l'entête des messages DNS, le format de conversion est le suivant : `!HHHHHH`. Le point d'exclamation indique que le flux d'octets est en big-endian. Les six H indiquent que le flux contient six entiers non signés de 16 bits. Les valeurs sont ensuite passées dans l'ordre de leur apparition dans l'entête.

Cette logique est utilisée par tous les objets qui doivent pouvoir être sérialisés (`DNSMessage`, `DNSQuestion`, `ResourceRecord`, `DomainName`).

### 3.1.3 Compression des noms et gestion de la section ADDITIONAL

Un nom de domaine est composé de plusieurs labels séparés par des points. Les labels sont lus de droite à gauche et chacun indique une sous-section de la zone définit par le précédent.

Les messages DNS contiennent souvent des domaines qui partagent des labels communs. En l'absence de compression ces répétitions provoqueraient une perte de bande passante et de temps de traitement considérable à l'échelle d'internet. Ainsi, à chaque fois qu'un label déjà présent dans le message doit être réutilisé, un pointeur de compression est utilisé à la place. Le pointeur de compression est un entier de 16 bits dont les deux premiers bits sont à 1 et les 14 autres indiquent l'offset du label dans le message.

Le compte des labels déjà présents dans le message est tenu par un dictionnaire (clé = label, valeur = offset) créer dans la fonction `pack` de la classe `DNSMessage` et passé à la fonction `pack` de `DomainName` pour chaque nom de domaine à sérialiser. À la lecture, une variable d'offset sert de tête de lecture pour savoir quel caractère est entrain d'être lu. Lorsqu'un pointeur est détecté par les deux bits à 1, l'offset est mis à jour avec la valeur du pointeur et la lecture continue à partir de cette position jusqu'à la fin du nom de domaine.

### 3.1.4 Encodage des enregistrements TXT

Contrairement aux adresses IP (types A et AAAA) qui ont une longueur fixe, les enregistrements de type TXT peuvent contenir des données textuelles de taille variable, souvent utilisées pour la vérification de domaine ou le filtrage anti-spam (SPF). La norme RFC 1035 impose que ces chaînes de caractères soient encodées sous la forme de `Pascal-strings`. Ainsi, la donnée brute n'est pas simplement du texte encodé en UTF-8, mais doit systématiquement être préfixée par un octet indiquant la longueur de la chaîne qui suit. De plus, ce format impose une contrainte stricte : un segment de texte ne peut excéder 255 octets. Si un enregistrement TXT est plus long, il doit être découpé en plusieurs blocs successifs (longueur + données) au sein du même `Resource Record`. Cette implémentation gère cette spécificité lors de la sérialisation pour garantir la compatibilité avec les clients standards.

## 3.2 Architecture réseau

### 3.2.1 Stratégie Réseau (UDP vs TCP)

Le protocole DNS repose historiquement sur le protocole UDP pour des raisons de vitesse. En effet, l'absence de poignée de main (`handshake`) permet des résolutions extrêmement rapides. Cependant, l'UDP étant sans connexion et non fiable, il limite la taille des messages à 512 octets.

Pour gérer les communications, l'architecture sépare strictement l'écoute (le serveur) et l'émission (le client). La classe `DNSListener` ouvre des `sockets` serveurs multi-threadées (un thread par requête TCP) pour ne pas bloquer la réception des paquets.

À l'inverse, la classe `DNSRequester` gère l'émission avec des délais d'attente (`timeouts`) stricts pour éviter qu'un serveur distant silencieux ne bloque notre résolveur.

### 3.2.2 Gestion du Fallback TCP

Lorsqu'un serveur DNS doit renvoyer une réponse dépassant la limite des 512 octets en UDP (par exemple, une réponse contenant de multiples enregistrements de type AAAA ou de nombreuses signatures), il tronque le message et met le drapeau `TC` (`Truncated`) à 1 dans l'entête du message. Notre résolveur implémente un mécanisme de `Fallback` transparent. Ainsi, s'il détecte `TC=1`, il abandonne la réponse incomplète et initie immédiatement une nouvelle connexion TCP vers le même serveur. La subtilité technique imposée par la RFC 1035 (section 4.2.2) réside dans l'encapsulation. C'est-à-dire que sur un flux TCP (qui est continu, contrairement aux datagrammes UDP), chaque message DNS doit obligatoirement être préfixé par 2 octets indiquant la taille totale de la trame. Notre classe `DNSRequester` gère l'ajout et le retrait de ce préfixe de manière transparente lors du basculement.

### 3.2.3 Générisité de l'architecture

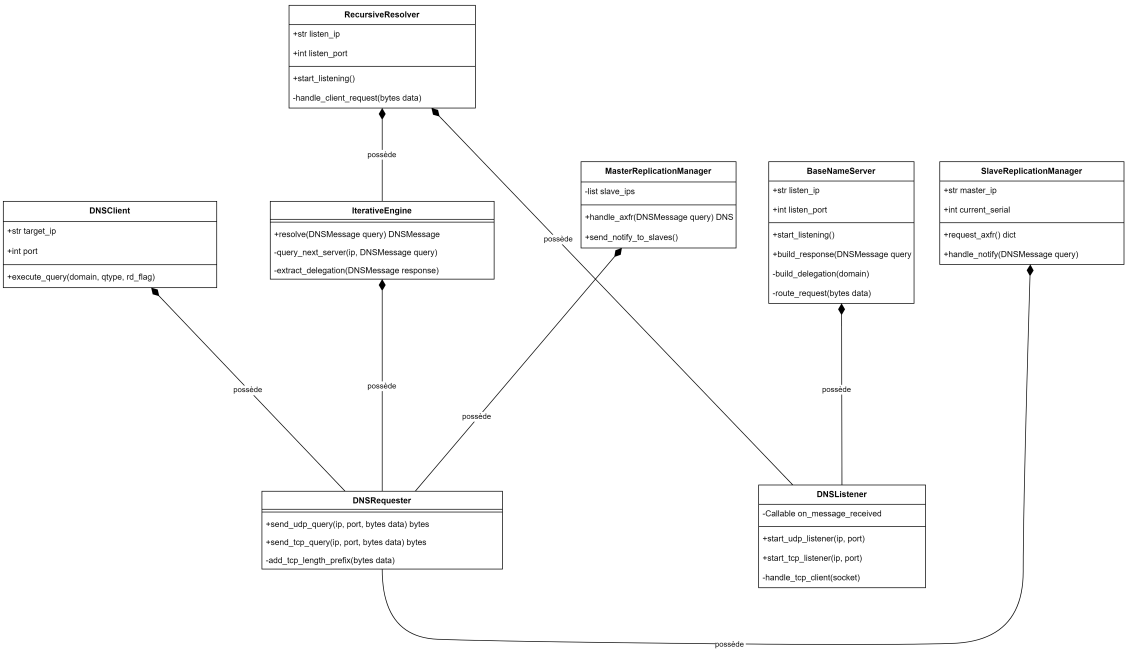



FIGURE 7 – Diagramme de classes de l'architecture réseau.

Comme illustré dans ce diagramme de classes, les classes `DNSListener` et `DNSRequester` sont utilisées par de nombreux composants du système. Il est donc essentiel que ces classes soient génériques et indépendantes de la logique métier.

La classe `DNSListener` permet d'ouvrir un serveur d'écoute, TCP ou UDP, sur un port donné. La réception des requêtes est entièrement gérée par cette classe. Une fois la requête interceptée, elle est transmise à une fonction de rappel (`callback`) fournie par le composant appelant.

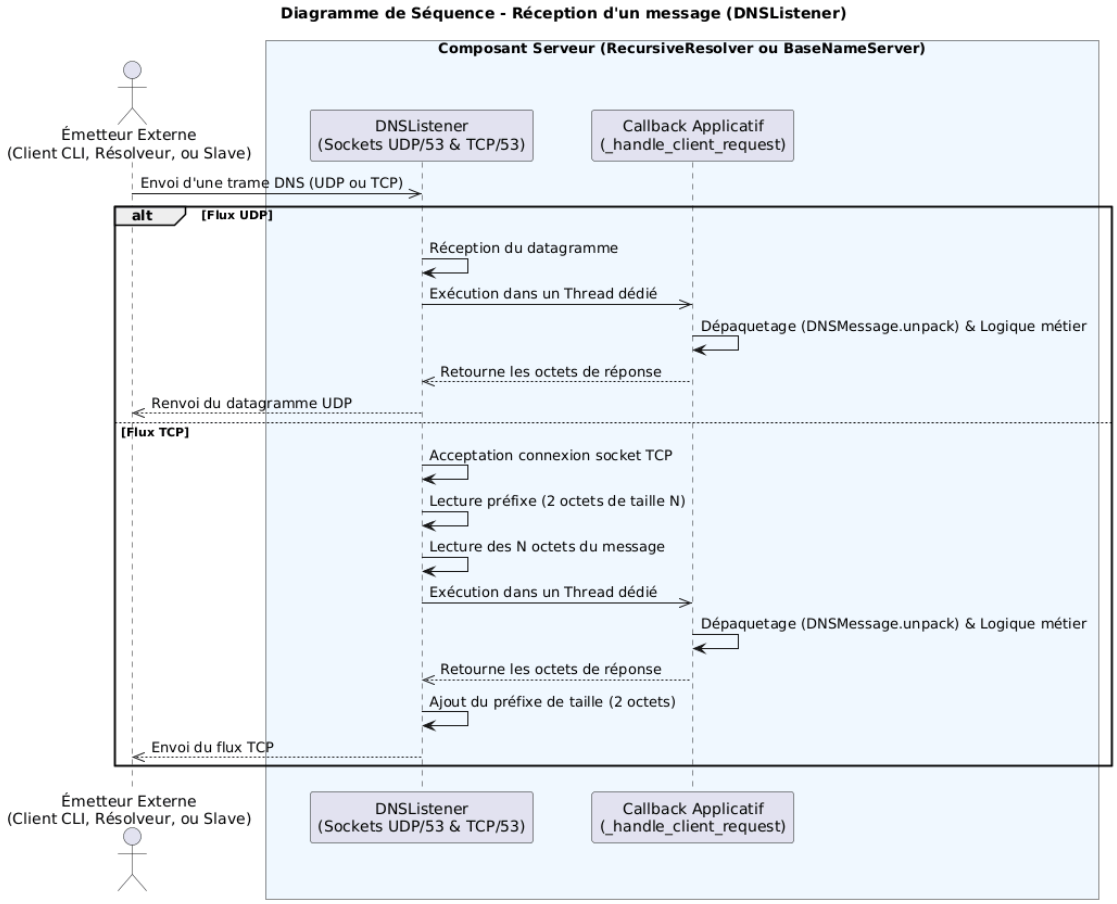



FIGURE 8 – Séquence de communication pour les requêtes entrantes.

La classe `DNSRequester` permet d'envoyer des requêtes DNS TCP et UDP à un serveur distant. Cette classe gère uniquement l'émission de la requête et la réception de la réponse. C'est le composant appelant qui est responsable de la création de la requête au format binaire et de l'analyse de la réponse reçue. Dans le cas d'une réponse tronquée, c'est le composant appelant qui décide de relancer la requête en TCP.

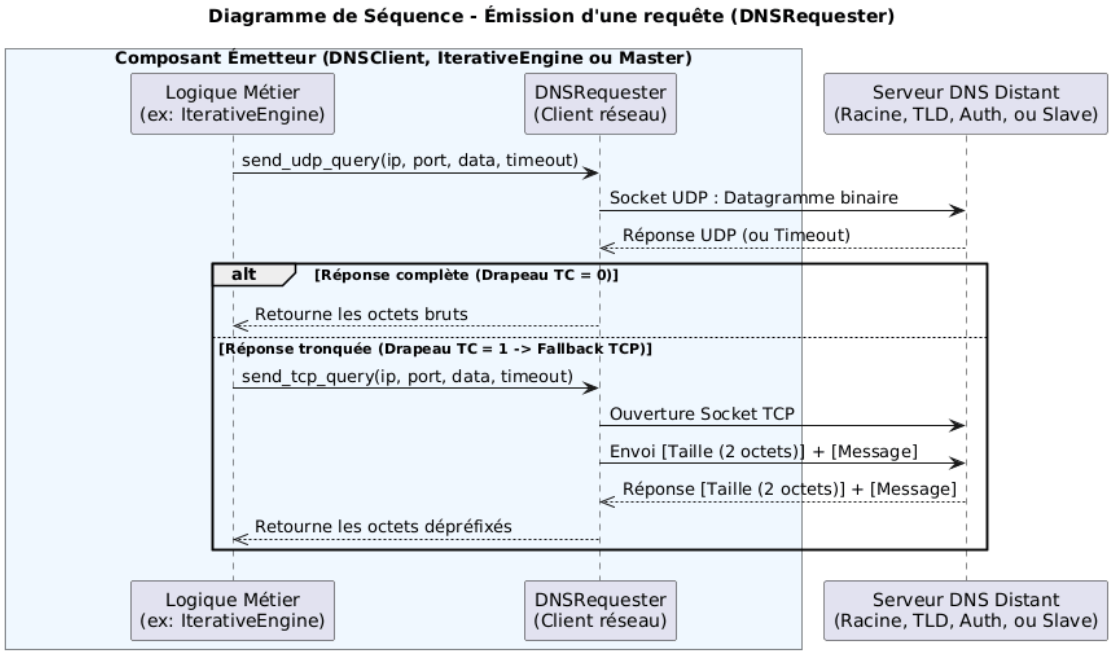



FIGURE 9 – Séquence de communication pour les requêtes sortantes.

## 3.3 Résolution DNS

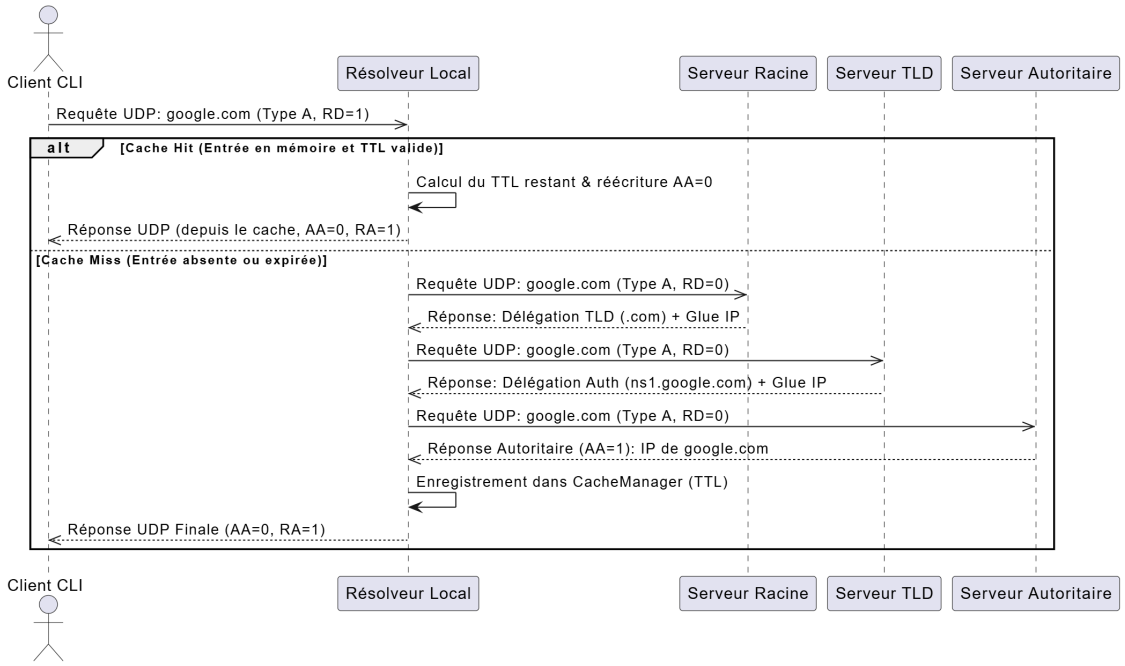



FIGURE 10 – Diagramme de séquence de la résolution.

### 3.3.1 Parcours d'une requête récursive

Le protocole DNS prévoit que l'information est répartie sur plusieurs serveurs de noms. Trouver le serveur autoritaire pour un nom de domaine donné nécessite souvent plusieurs étapes. Il faut d'abord interroger les serveurs Racines pour obtenir les serveurs de noms des TLD, puis interroger ces derniers pour obtenir les serveurs de noms spécifiques à la zone, et enfin interroger le serveur autoritaire pour obtenir la réponse finale. La résolution récursive est le processus par lequel un résolveur DNS effectue ces étapes de manière transparente pour le client.

Le cœur de la résolution récursive réside dans la classe `IterativeEngine`. Lorsqu'un client demande la résolution d'un nom de domaine avec le flag `RD=1` (`Recursion Desired`), le moteur entame une boucle itérative en interrogeant d'abord les serveurs Racines (`Root Hints`). La logique repose sur l'analyse de la réponse à chaque itération.

La condition d'arrêt de cette boucle est régie par un opérateur logique `OU` (OR) vérifiant trois critères :

1. `response.answers` n'est pas vide : La réponse finale a été trouvée.
2. `response.rcode != 0` : Le serveur distant signale une erreur définitive (ex : NXDOMAIN, indiquant que le domaine n'existe pas).
3. `response.aa == 1` : Le serveur interrogé fait autorité sur la zone, même s'il ne renvoie pas de réponse (cas d'un `NODATA`, indiquant que le domaine existe mais pas pour le type

demandé).

Tant qu'aucune de ces conditions n'est remplie, le résolveur extrait les adresses IP (`Glue Records`) situées dans la section `Additional` de la réponse pour identifier le prochain serveur d'autorité à interroger (TLD, puis serveur de noms spécifique) et relance la boucle.

### 3.3.2 Gestion du Cache

Une résolution DNS récursive peut générer un grand nombre de requêtes et de réponses. Il est donc nécessaire de stocker les réponses reçues pour éviter de surcharger le réseau avec des requêtes redondantes. Un système de cache situé au niveau du résolveur récursif permet d'éviter cette surcharge.

Lorsque le résolveur récursif reçoit une requête, il vérifie si la réponse est déjà présente dans le cache. Si c'est le cas, il renvoie la réponse directement au client sans interroger les serveurs de noms. S'il n'y a pas de réponse stockée pour cette requête ou que cette dernière a expirée, le résolveur procède avec une résolution récursive classique.

Ce mécanisme permet de réduire le temps de réponse pour les clients et d'alléger la charge sur les serveurs de noms. Cependant, en ne vérifiant pas les réponses au près d'une source autoritaire, il est possible que le cache contienne des informations obsolètes. C'est pour cela que chaque enregistrement DNS possède un champ TTL (Time To Live) qui indique combien de temps il peut être conservé dans le cache avant d'être considéré comme périmé. Le TTL est défini par l'administrateur de la zone et est transmis dans les réponses DNS.

Le système de cache est implémenté dans une classe indépendante pour garantir la modularité (il devient facile d'ajouter ce mécanisme à d'autres composants comme le client). Cette classe, `CacheManager`, a comme attribut un dictionnaire et comme méthodes `store`, `retrive` et `clear_cache`. Le dictionnaire a comme clé une combinaison du nom de domaine et type de requête et comme valeur, un objet contenant le message de réponse sauvegardé pour cette requête et le temps d'expiration (TTL). Si le résultat de la requête est une erreur (NXDOMAIN, SERVFAIL, REFUSED), il est également stocké dans le cache avec un TTL minimal (300s) pour éviter de répéter la même requête inutilement.

Les données du cache sont stockées en mémoire vive pour un accès rapide. Cela implique que le cache est perdu à chaque redémarrage du résolveur récursif. En effet, il n'y a pas de bénéfice à stocker les réponses dans un fichier sur le disque dur puisque le TTL des enregistrements est généralement très court (de quelques secondes à quelques heures) et que le cache se reconstruit rapidement à l'utilisation.

Lors du stockage, une copie profonde du message de réponse est ajoutée au dictionnaire avec le temps d'expiration calculé en ajoutant le TTL de la réponse au temps actuel lors du stockage.

Lors de la récupération, la validité de la réponse est vérifiée, son id est remplacé par celui de la requête et le flag AA est mis à 0 pour indiquer que la réponse provient d'un cache non autoritaire. Le TTL restant doit également être recalculé en soustrayant le temps actuel du temps d'expiration.

Enfin, la méthode `clear_cache` permet de vider le cache en supprimant toutes les entrées du

dictionnaire. Elle peut être déclenchée par le client en envoyant une requête de type `TXT` avec le nom de domaine `flush.cache.local`.

## 3.4 Chargement de zones

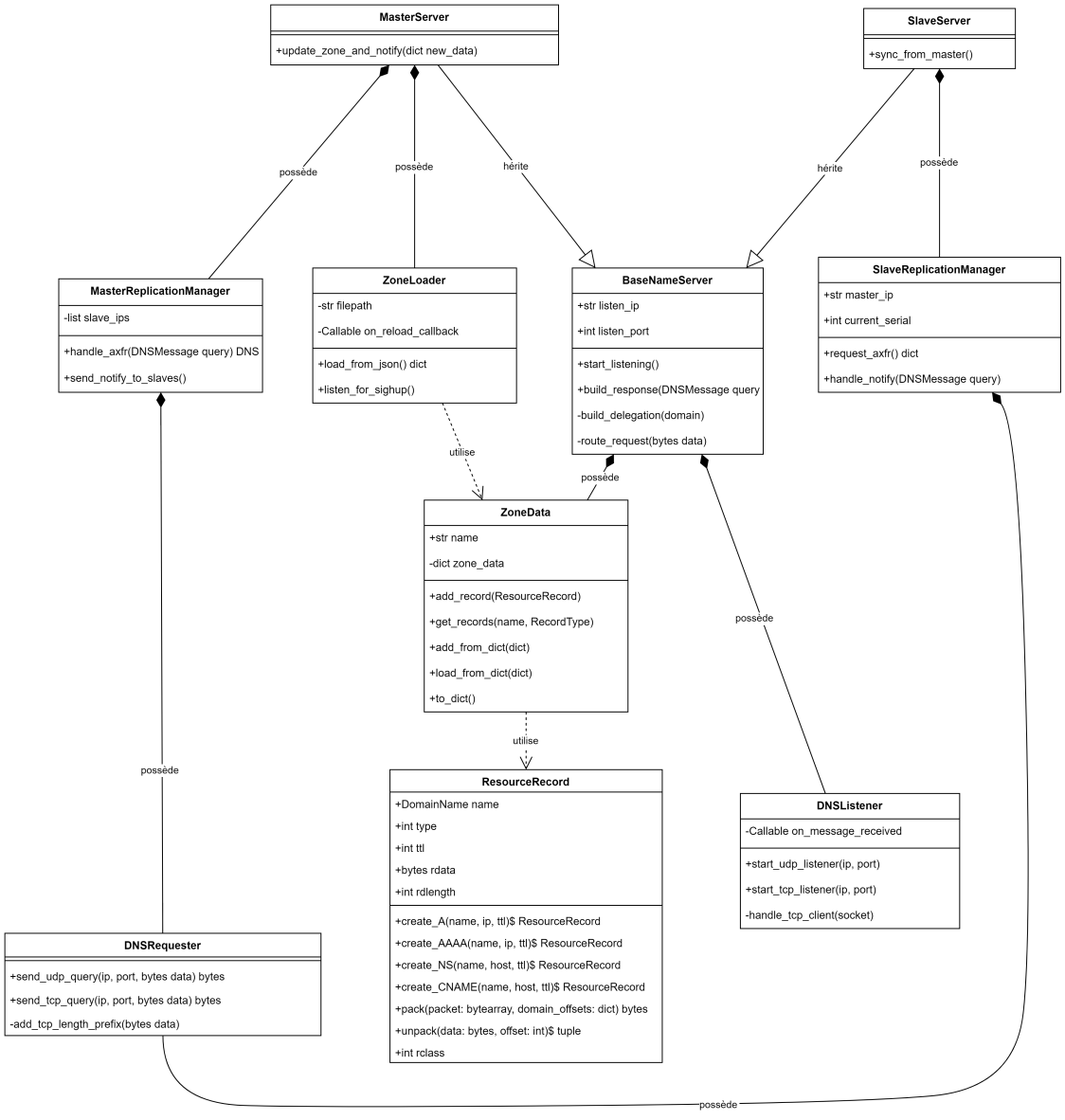



FIGURE 11 – Diagramme de classes du chargement de zones.

### 3.4.1 Stockage JSON

Pour qu'un serveur de nom puisse répondre aux requêtes, il doit posséder les informations de la zone qu'il gère. Alors que la norme historique (RFC 1035) préconise l'utilisation de fichiers de zone au format texte brut BIND (caractérisé par des directives telles que `$ORIGIN` ou des notations relatives avec `@`), nous avons fait le choix de stocker les zones dans des fichiers au format JSON.

Le format retenu structure la zone sous la forme d'un dictionnaire imbriqué où chaque nom de domaine constitue une clé de premier niveau, associée à un sous-dictionnaire indexé par type d'enregistrement (`A`, `AAAA`, `NS`, `TXT`, etc.) :

```
{
  "be": {
    "NS": [
      {"host": "ns1.tld.be", "ttl": 86400}
    ]
  },
  "ns1.tld.be": {
    "A": [
      {"ip": "10.53.0.101", "ttl": 300}
    ]
  }
}
```

Ce choix s'appuie sur plusieurs arguments techniques et pragmatiques :

— **Simplicité et robustesse du parsing :** Le format BIND traditionnel nécessite l'écriture d'un analyseur syntaxique (`parser`) complexe pour gérer les retours à la ligne, les directives relatives et les commentaires. À l'inverse, le format JSON est nativement pris en charge par le module `json` de la bibliothèque standard de Python. Son chargement est immédiat et évite d'introduire des bogues de lecture sur le serveur.

— **Adéquation avec la structure en mémoire :** La hiérarchie du fichier JSON reflète exactement la structure de données interne utilisée par notre classe `ZoneData`. Cette correspondance directe permet d'instancier les objets `ResourceRecord` sans transformation lourde et garantit des recherches d'enregistrements en temps constant $\mathcal{O}(1)$ grâce aux tables de hachage.

— **Indépendance vis-à-vis de l'interopérabilité réseau :** Le fichier de zone est un composant strictement local et interne au serveur d'autorité. Son format sur disque n'impacte en rien le protocole réseau : toutes les trames échangées avec l'extérieur (UDP et TCP) respectent scrupuleusement la norme binaire RFC 1035. Les clients standards (`dig`, `nslookup`) ou les résolveurs tiers interagissent avec le serveur sans jamais avoir connaissance du format de stockage interne.

— **Évolutivité via la classe ZoneData :** La gestion du stockage est entièrement isolée au sein de la classe `ZoneData`. Si une compatibilité avec les fichiers de zone BIND historiques s'avérait nécessaire à l'avenir, il suffirait d'ajouter une méthode de traduction/parsing BIND dans cette classe sans devoir modifier le moteur du serveur ni le reste de l'architecture.

### 3.4.2 Rechargement à chaud et surveillance de fichier

Afin de répondre au besoin de continuité de service (US-3.6), le serveur d'autorité doit pouvoir intégrer des modifications de la zone DNS sans nécessiter un redémarrage du processus. Initialement, cette fonctionnalité a été pensée autour de l'interception du signal UNIX `SIGHUP` via le module `signal`. Cependant, ce signal n'étant pas nativement supporté par le système d'exploitation Windows, une approche plus universelle a été adoptée pour garantir la portabilité du code. L'architecture finale repose sur un thread d'arrière-plan (`background thread`) exécutant une boucle de `polling`. Ce thread interroge périodiquement les métadonnées du fichier de zone via `os.path.getmtime()` pour détecter un changement de date de modification. Si une altération est détectée, le fichier est rechargé, le dictionnaire en RAM est écrasé de manière `thread-safe` (utilisant un verrou de synchronisation), et une notification de mise à jour est déclenchée.

### 3.4.3 La Réplication Master / Slave et Rechargement

Pour assurer la redondance et la tolérance aux pannes (Épic 4), un mécanisme de réplication a été implémenté entre un serveur `Master` et des serveurs `Slaves`. Dès que le serveur Master détecte une modification de sa zone, il émet une trame UDP avec l'OpCode `NOTIFY` (4) à destination de ses Slaves. À la réception de ce signal, le Slave initie une connexion TCP pour demander un transfert de zone complet (requête de type `AXFR`, QTYPE 252).

La norme RFC 1035 décrit traditionnellement le transfert AXFR comme un flux séquentiel de multiples enregistrements binaires encadré par des enregistrements SOA. Cependant, étant donné que notre écosystème stocke nativement les zones au format JSON, nous avons opté pour une approche pragmatique et simplifiée. Le serveur Master sérialise l'intégralité du dictionnaire JSON en une seule chaîne de caractères encodée en UTF-8, et l'encapsule dans la section `RDATA` d'un unique enregistrement de type 252. Le Slave reçoit cette trame TCP, extrait le JSON, et met à jour sa mémoire instantanément. Cette méthode respecte l'encapsulation DNS tout en tirant pleinement parti des structures de données de notre implémentation.

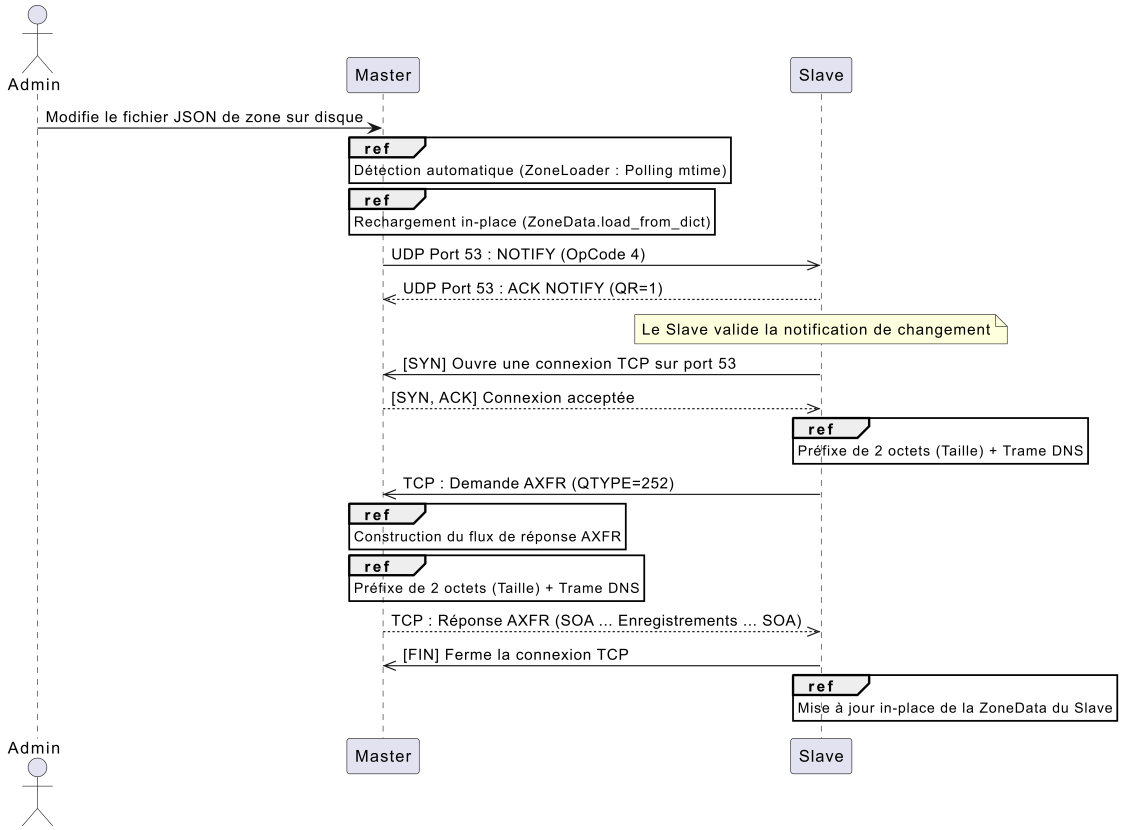



FIGURE 12 – Diagramme de séquence de la réplication.

## 3.5 Conception des classes

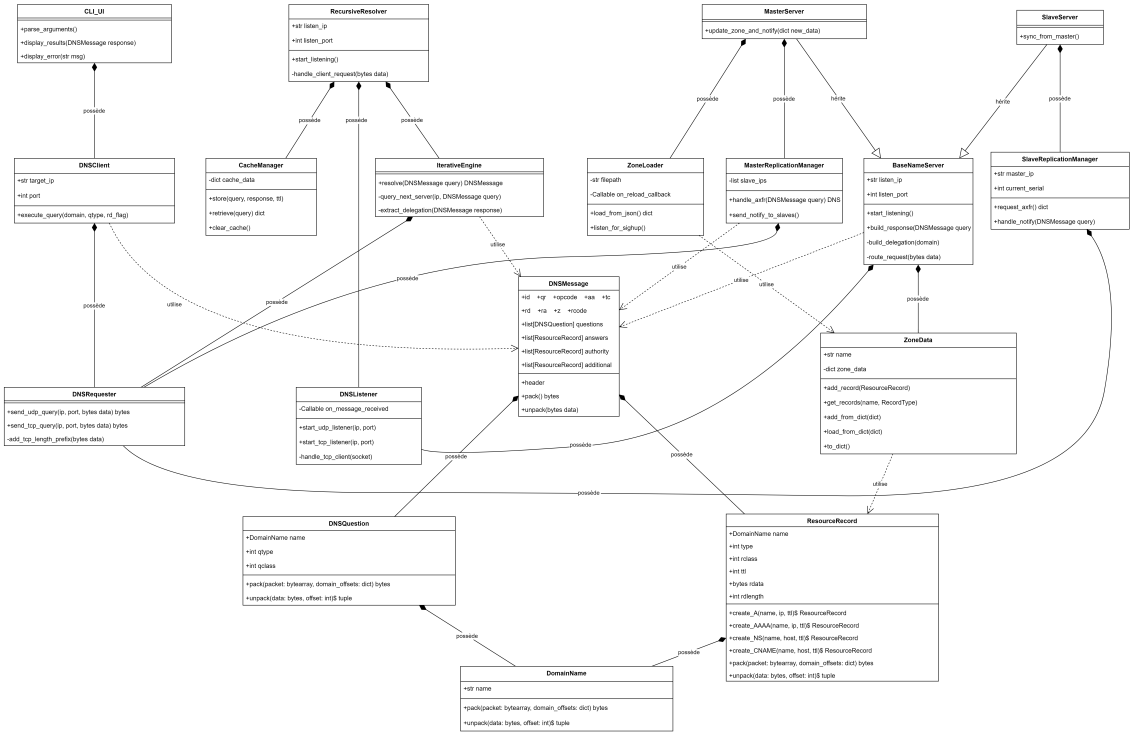



FIGURE 13 – Diagramme de classes.

# 4 Implémentation

## 4.1 Arborescence du Projet

Afin d'éviter d'effectuer des restructurations successives, ce qui est mauvais pour l'historique Git, l'arborescence du projet a été pensée dès le départ pour séparer les différents composants du projet. Ainsi, la documentation a été séparée du code source, lui-même séparé des tests et de l'infrastructure de déploiement. Le code source et les tests sont eux séparés en plusieurs modules : `client`, `resolveur`, `serveur de noms` et `core` qui contient les classes de base et les utilitaires partagés par les autres modules.




FIGURE 14 – Arborescence du projet.

## 4.2 Qualité et Tests (TDD)

Afin de garantir la fiabilité du système, le respect de la RFC 1035 et la non-régression lors des phases de refactoring, le développement a été guidé par des tests automatisés implémentés à l'aide de `pytest`. La suite de tests comprend plus de 39 tests unitaires et d'intégration couvrant l'ensemble des modules de l'application, du parsing binaire jusqu'à la réplication Master/Slave.

### 4.2.1 Tests unitaires

La strucutre de fichiers des tests unitaires immite celle du code source (`tests/core/`, `tests/client/`, `tests/name_server/`, `tests/recursive_resolver/`) de manière à isoler et de valider chaque composant indépendamment :

**Isolation réseau et simulation des Sockets :**
Grâce au module `unittest.mock`, les classes réseau telles que `DNSListener` et `DNSRequester` sont testées sans ouvrir de vrais ports système ni dépendre d'une connectivité extérieure. Les méthodes `sendto`, `recvfrom`, `recv` et `sendall` des sockets sont simulées pour vérifier le bon découpage des datagrammes UDP, la lecture du préfixe de longueur de 2 octets en TCP, ainsi que la gestion des exceptions (`socket.timeout`, `ConnectionRefusedError`).

**Validation de la sérialisation binaire :**
Les tests de `DNSMessage` contrôlent l'exactitude du (dé)paquetage des en-têtes (masques de drapeaux bit-à-bit pour `QR`, `Opcode`, `AA`, `TC`, `RD`, `RA`, `Rcode`), la sérialisation des quatre sections (*Question*, *Answer*, *Authority*, *Additional*), la création des différents enregistrements (`A`, `AAAA`, `NS`, `CNAME`, `TXT`, `AXFR`) et l'efficacité de la compression de noms par pointeurs (`0xC0`).

**Simulation du temps et expiration du cache :**
La classe `CacheManager` fait l'objet de tests dédiés où le temps système est manipulé artificiellement via `patch("time.time")`. Cela permet de valider sans délai d'attente réel le recalcul dynamique du TTL restant, l'effacement automatique des enregistrements périmés (*Lazy Expiration*), la réécriture du drapeau `AA=0`, ainsi que la sensibilité à la casse des noms de domaine.

**Stratégie de test des threads d'arrière-plan et rechargement à chaud :**
La vérification de la surveillance des fichiers de zone dans `ZoneLoader` repose sur une stratégie combinant des fichiers temporaires et la modification programmée de leurs métadonnées système. Le thread de surveillance est validé soit en substituant `threading.Thread` par un mock, soit en modifiant physiquement la date de dernière modification du fichier de zone à l'aide de `os.utime()` pour simuler une sauvegarde sur disque et déclencher l'appel automatique du callback de rechargement.

### 4.2.2 Tests de bout en bout

Les tests d'intégration et de bout en bout (`tests/end-to-end/`) valident le comportement global du système en instanciant simultanément l'ensemble des acteurs au sein du même processus de test :

**Stack de résolution récursive complète (`test_full_dns_stack.py`) :**// Ce test monte une

architecture DNS complète comprenant un serveur Racine, un serveur TLD (.be), un serveur Autoritaire et un Résolveur Récursif. Pour éviter tout conflit de port lors de l'exécution automatisée, une fonction utilitaire (`_free_port()`) attribue dynamiquement des ports éphémères libres sur la boucle locale (`127.0.0.1`). Un mécanisme de sonde (`_probe_server()`) attend que chaque serveur soit prêt avant d'émettre les requêtes. Le scénario valide :

1. La résolution itérative complète pour un enregistrement `A` et `AAAA` à travers toute la hiérarchie.
2. Le comptage exact des requêtes reçues par chaque serveur pour prouver que la seconde requête identique est servie directement depuis le cache du résolveur sans solliciter les serveurs amont.
3. L'interception et le bon traitement de la commande d'administration `flush.cache.local` (type `TXT`) qui vide le cache et force une nouvelle résolution itérative lors de la requête suivante.

**Réplication Master/Slave et rechargement (`test_master_slave_replication.py`) :**// Ce test simule le cycle de vie complet de la synchronisation entre un serveur `MasterServer` et un serveur `SlaveServer`. Il vérifie que lors d'un appel à `on_zone_reload()`, le Master envoie une notification `NOTIFY` en UDP au Slave, qui réagit immédiatement en initiant un transfert de zone `AXFR` en TCP sur le port du Master pour mettre à jour sa propre `ZoneData` locale et incrémenter son numéro de série.

## 4.3 Bugs critiques identifiés et résolus

Lors des phases de tests d'intégration et d'interopérabilité avec des outils système réels (`dig`, `nslookup`), plusieurs dysfonctionnements complexes sont apparus :

**Échecs de comparaison silencieux sur le typage des Enums :**
Lors de la recherche des délégations dans `ZoneData`, les comparaisons entre la chaîne de caractères brute `"NS"` et l'énumération `RecordType.NS` renvoyaient systématiquement `False` sans lever d'erreur. Ce bogue de typage entraînait l'échec silencieux de la détection des serveurs autoritaires délégués et provoquait un repli erroné vers une réponse `NXDOMAIN`. La correction a consisté à appliquer un typage strict et à systématiser la conversion vers les objets `RecordType` lors des requêtes d'indexation dans les dictionnaire de zone.

**Piège de la référence mémoire lors du rechargement à chaud (*Hot-Reload*) :**
Initialement, le rappel (*callback*) du composant `ZoneLoader` réinstanciait un nouvel objet `ZoneData` lors de la détection d'une modification de fichier JSON. Cependant, l'`AuthoritativeEngine` conservait une référence vers l'ancienne instance en mémoire RAM. Cette erreur de conception menait à un comportement incohérent. En effet, bien que les logs indiquaient un rechargement réussi, le serveur continuait de traiter les requêtes avec la zone obsolète, provoquant des incohérences dans les réponses DNS. Ce problème a été résolu en implémentant une mise à jour *in-place* via la méthode `load_from_dict()`, permettant de vider et de réapprovisionner l'instance existante sans rompre les liaisons d'objets au sein du moteur d'exécution.

**Gestion des enregistrements non pris en charge (EDNS0 / Type 41) :**

Les utilitaires clients modernes comme `dig` ou `nslookup` injectent fréquemment un enregistrement pseudo-ressource de type `OPT` (Type 41, lié à l'extension EDNS0) dans la section `ADDITIONAL`. Lors de la désérialisation (*unpacking*) de la trame binaire, le parseur levait une exception non interceptée face à ce type non implémenté, interrompant brutalement le traitement et renvoyant une réponse vide qui provoquait un *timeout* côté client. La logique de lecture des *Resource Records* a été assouplie pour que les enregistrements de type inconnu ou non supporté soient simplement ignorés en levant *warning* dans les logs. De cette manière, une compatibilité totale avec les outils d'interrogation standards est rétablie.

## 4.4 Déploiement sous Docker

Le projet est déployé dans un environnement *Docker Compose*, permettant de simuler un réseau DNS complet avec des serveurs de noms et un résolveur récursif. De cette manière, il est possible d'essayer le code en instanciant plusieurs serveurs sur une machine unique.

Étant donné que tout le projet a été développé en Python, en utilisant uniquement la librairie standard, il n'y a pas de dépendances externes à installer. Un seul *Dockerfile* est donc suffisant pour construire l'image de tous les composants du projet. En effet, le docker file utilise l'image officielle `python:3.11-slim` et copie le code source dans le conteneur. Chaque serveur peut ensuite être lancé en exécutant la commande adaptée. Les fichiers de configuration et de zones sont montés dans le conteneur via des volumes Docker, ce qui permet de modifier les fichiers sur l'hôte et de voir les changements immédiatement dans le conteneur.

Cette structure est hautement modulaire et permet de tester n'importe quel configuration en ne modifiant que le fichier `docker-compose.yml` les fichiers de configuration et les fichiers de zones.

La configuration choisie pour répondre à la demande du client est la suivante :

— Un serveur de noms racine (`root`) qui contient les délégations vers les serveurs de noms autoritaires.

— Un serveur de noms top-level (`tld`) qui contient les délégations vers les serveurs de noms autoritaires pour le domaine `example.com`.

— Un serveur de noms autoritaire (`authoritative`) qui contient les enregistrements DNS pour le domaine `example.com`.

— Un résolveur récursif (`resolver`) qui interroge les serveurs de noms pour résoudre les requêtes DNS. Le port d'écoute du résolveur récursif est configuré sur le port 10054 pour éviter les conflits avec le port 53 utilisé par le système d'exploitation de l'hôte.

— Un serveur assurant la redondance du serveur de noms racine.

— Le client qui permet d'interroger le résolveur récursif ne fait pas partie du réseau DNS simulé sur docker.Il peut, cependant, être utilisé pour l'interroger directement sur le port 10054.

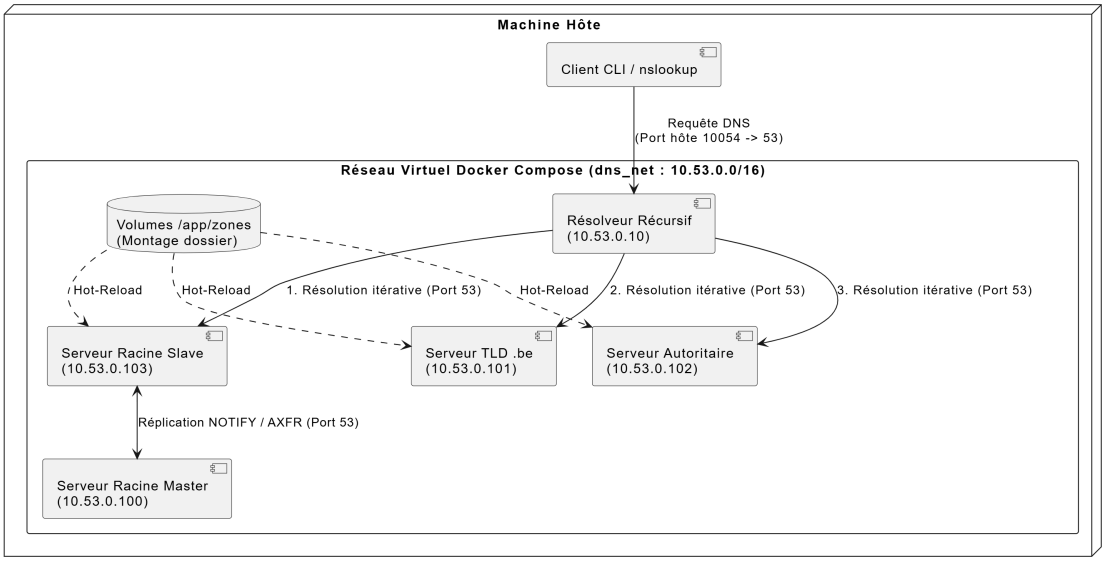



FIGURE 15 – Architecture Réseau de l'Environnement de Test Docker Compose.

# 5 Analyse retrospective

## 5.1 Bilan des résultats

L'évaluation des résultats s'appuie directement sur les exigences du cahier des charges initial. L'ensemble des fonctionnalités demandées a été implémenté, testé et validé sur notre infrastructure :

- **Chaîne de résolution de noms :** L'infrastructure de résolution globale est pleinement opérationnelle. Le résolveur récursif (`RecursiveResolver`) orchestre le parcours itératif en interrogeant successivement le serveur Racine, le serveur TLD (gestion de l'extension `.be`) et le serveur autoritaire responsable du nom de domaine complet et de ses sous-domaines (ex : `www.test.be`).
- **Redondance du serveur Racine :** La continuité de service est assurée par un serveur racine secondaire (`SlaveServer`) qui communique directement avec le serveur racine principal (`MasterServer`). La synchronisation s'effectue via les mécanismes standards `NOTIFY` (UDP) et `AXFR` (TCP), garantissant le maintien de la zone même en cas de panne du master.
- **Support des types A / AAAA et gestion du cache :** Chaque serveur et résolveur gère nativement la résolution des enregistrements d'adresses IPv4 (`A`) et IPv6 (`AAAA`) sur de multiples domaines. La zone de cache configurable (`CacheManager`) est intégrée et respecte les durées de vie (`TTL`) propres à chaque enregistrement et serveur pour optimiser les performances.
- **Outil client en ligne de commande (CLI) :** Conformément aux recommandations, l'outil client (`DNSClient` / `cli_ui.py`) a été développé dès le début du projet. Il a servi d'outil de diagnostic continu pour valider la forge des paquets et permet d'émettre des requêtes personnalisées (vidage du cache).
- **Utilisation des protocoles UDP / TCP (Sans API REST) :** Toutes les communications entre le client, le résolveur et les serveurs de noms reposent exclusivement sur des sockets UDP et TCP sur les ports dédiés (53 et 10054). Aucune interface HTTP ou API REST n'a été employée, garantissant le respect de la norme RFC 1035.
- **Compatibilité avec les outils système standards :** En exposant le port du résolveur récursif (port `10054`) au niveau de l'orchestration Docker, tout ordinateur personnel hôte peut utiliser la solution via une simple configuration réseau ou l'utilisation d'utilitaires système standards (`nslookup`, `dig`).

## 5.2 Limites et perspectives d'améliorations

Malgré la conformité et la stabilité de l'implémentation, certaines pistes d'amélioration peuvent être envisagées :

- **Modèle de concurrence :** L'architecture réseau actuelle de `DNSListener` repose sur un modèle multi-threadé synchrone (*Thread-per-connection* pour le TCP). Bien que parfaitement adapté à notre environnement de test et à des charges modérées, ce modèle montre ses

limites face à des volumes massifs de requêtes ou des attaques par déni de service (DDoS) en raison du coût d'instanciation des threads. Une refonte vers une architecture asynchrone guidée par les événements (*Event Loop* via le module `asyncio` de Python) constituerait une évolution majeure pour la montée en charge.

— **Support protocolaire avancé (EDNS0 et Path MTU) :** Afin de prévenir tout plantage lors de l'interrogation par des outils système réels (`dig`, `nslookup`), le système tolère et ignore la section `ADDITIONAL` contenant des pseudo-enregistrements `OPT` (Type 41). Cependant, l'extension EDNS0 (RFC 6891) n'est pas pleinement implémentée. La prise en charge native d'EDNS0 permettrait de négocier des tailles de tampons UDP supérieures à 512 octets et d'optimiser le transfert de réponses volumineuses sans basculer de manière systématique sur TCP.

— **Persistance du cache DNS :** Le cache du résolveur récursif réside exclusivement en mémoire RAM volatile. Bien que suffisant pour un environnement de test et justifié par l'analyse technique, le client pourrait apprécier la persistance du cache sur disque pour éviter un démarrage à froid et conserver les enregistrements non expirés entre deux redémarrages.

— **Sécurisation du protocole (DNSSEC) :** En l'état, le résolveur accorde une confiance aveugle aux réponses fournies par les serveurs de noms sans vérifier l'authenticité de la source ni l'intégrité des données. L'implémentation des extensions de sécurité DNSSEC (RFC 4033 à 4035) permettrait de protéger le système contre l'empoisonnement de cache (*DNS Cache Poisoning*). Cette implémentation nécessiterait la validation des chaînes de confiance cryptographiques au moyen d'enregistrements `RRSIG`, `DNSKEY`, `DS` et `NSEC/NSEC3`.

— **Non-conformité stricte de la réplication AXFR :** Comme argumenté dans l'analyse technique, le mécanisme de transfert de zone (`AXFR`) s'appuie sur une encapsulation applicative du dictionnaire JSON complet au sein du champ `RDATA` d'un unique enregistrement de type 252. Bien que cette solution soit parfaitement fonctionnelle, élégante et justifiée au sein de notre écosystème fermé, elle ne respecte pas le protocole binaire standard de la RFC 1035 (qui exige un flux continu d'enregistrements binaires encadré par deux enregistrements `SOA`). Un alignement strict sur la norme permettrait une interopérabilité directe avec des serveurs secondaires tiers.

## Déclaration d'usage de l'IA générative

Conformément au Guide du Projet Innovant de l'Hénallux, je déclare avoir utilisé le grand modèle linguistique Gemini (Google) en tant qu'assistant à la rédaction lors de ce projet. Son usage s'est limité à la reformulation académique de textes préalablement rédigés par mes soins, la mise en forme du code *latex* et à l'aide à la structuration des sections du rapport. L'entièreté de la réflexion, des choix techniques et de l'analyse sont le fruit de mon travail personnel.

Google, "Gemini," May 2024. [Large language model]. Available : https ://gemini.google.com/

# Table des figures


1 Architecture globale des flux entre le Client, le Résolveur et la Hiérarchie DNS. . 7

2 Diagramme de cas d'utilisation du Client CLI. . . . . 9

3 Diagramme de cas d'utilisation du Résolveur Récursif. . . . . 10

4 Diagramme de cas d'utilisation du Serveur de Nom. . . . . 11

5 Format des messages DNS [1]. . . . . 13

6 Diagramme de classe de la Forge à messages. . . . . 14

7 Diagramme de classes de l'architecture réseau. . . . . 17

8 Séquence de communication pour les requêtes entrantes. . . . . 18

9 Séquence de communication pour les requêtes sortantes. . . . . 19

10 Diagramme de séquence de la résolution. . . . . 20

11 Diagramme de classes du chargement de zones. . . . . 22

12 Diagramme de séquence de la réplication. . . . . 25

13 Diagramme de classes. . . . . 26

14 Arborescence du projet. . . . . 27

15 Architecture Réseau de l'Environnement de Test Docker Compose. . . . . 31